PROJET DE RENTRÉE ESIS-2 / CPDIA-2

Assistant Virtuel Éducatif ESIC

Guide d'explication du code — Sprint 1 (Chatbot MVP)

Contenu de ce guide

- La méthode de développement, étape par étape
- L'explication de chaque fichier du dépôt GitHub
- Les définitions de tous les termes techniques
- Les tests réalisés et les limites connues
- Un quiz d'auto-évaluation avec son corrigé

Technologies : Python, Flask, scikit-learn, PostgreSQL, React, Vite, Docker

Dépôt : github.com/guettafikrame-boop/Projet-chatbot-ikrame-

Sommaire

1. La méthode : comment aborder un projet
2. La structure du dépôt
3. Le nettoyage des données — `scripts/clean_data.py`
4. La base de données — `db/init.sql`
5. Le backend Python — dossier `backend/`
6. Le frontend React — dossier `frontend/`
7. L'assemblage — `docker-compose.yml`
8. Les petits fichiers (.gitignore, README…)
9. Les tests réalisés
0. Les limites et la suite (Sprint 2)
1. Lancer le projet sur ton Mac
2. Glossaire
3. Quiz d'auto-évaluation et corrigé

1. La méthode : comment aborder un projet

Le code n'a pas été écrit au hasard. Il suit l'ordre des **dépendances** : chaque étape s'appuie sur la précédente.

#	Étape	Résultat
1	Comprendre le besoin	Lecture du cahier des charges, repérage des User Stories (US-001 à US-013 pour le Sprint 1)
2	Comprendre les données	Ouverture de chaque fichier, détection des défauts
3	Nettoyer les données	`scripts/clean_data.py`
4	Prévoir le stockage	`db/init.sql` (table des logs)
5	Écrire le « cerveau »	Dossier `backend/` en Python
6	Tester le cerveau seul	Questions de test, réglage des seuils
7	Écrire l'interface	Dossier `frontend/` en React
8	Tout assembler	`docker-compose.yml`
9	Tester de bout en bout	Appels à l'API + captures d'écran
10	Documenter et sauvegarder	`README.md`, commit, push sur GitHub

Règle d'or. On teste chaque brique séparément avant de les assembler. Si on assemble tout d'un coup et que ça ne marche pas, on ne sait pas d'où vient l'erreur.

Le vocabulaire du cahier des charges

Sprint : période courte (ici 1 semaine) au bout de laquelle on livre une version qui fonctionne. C'est le principe de la méthode **Agile**.

MVP (Minimum Viable Product) : la version la plus simple possible qui marche déjà. On la livre vite, puis on l'améliore.

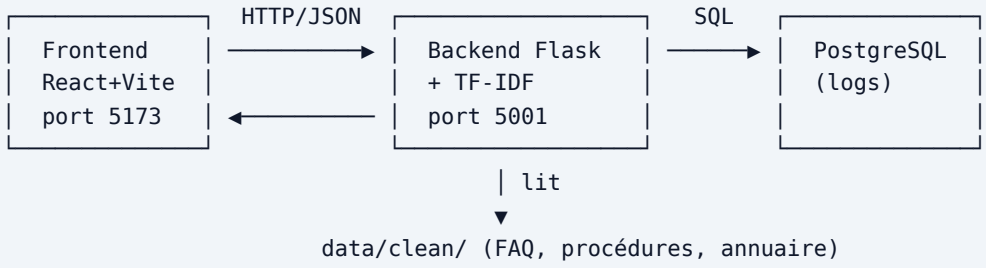
User Story (US) : un besoin écrit du point de vue de l'utilisateur : « En tant que… je veux… ». **Epic** : un groupe de User Stories sur le même thème.

2. La structure du dépôt

```
Projet-chatbot-ikrame-/
├── data/
│   ├── raw/            ← données d'origine (jamais modifiées)
│   └── clean/          ← données réparées, lues par le chatbot
├── scripts/
│   └── clean_data.py   ← la recette qui transforme raw → clean
├── db/
│   └── init.sql        ← création de la table dans PostgreSQL
├── backend/            ← le serveur Python (le cerveau)
│   ├── app.py          ← les routes de l'API
│   ├── chatbot.py      ← la décision : que répondre ?
│   ├── knowledge_base.py ← chargement des données + recherche
│   ├── security.py     ← validation, anti-injection, limite de débit
│   ├── database.py     ← enregistrement dans PostgreSQL
│   ├── requirements.txt← dépendances Python
│   └── Dockerfile      ← recette du conteneur Python
├── frontend/           ← la page web React
│   ├── src/App.jsx     ← l'interface du chat
│   ├── src/TexteFormate.jsx ← affichage sécurisé des réponses
│   ├── src/App.css     ← le style
│   └── Dockerfile      ← recette du conteneur Node
├── docs/               ← cahier des charges, guides
├── docker-compose.yml  ← lance les 3 conteneurs ensemble
├── .gitignore          ← fichiers à NE PAS envoyer sur GitHub
└── README.md           ← la vitrine du projet
```

Séparation des responsabilités : un dossier = un rôle, un fichier = un rôle. Pour changer la couleur des bulles, on va dans `frontend/` ; pour changer la logique de réponse, dans `backend/`. On ne se perd pas, et on peut modifier une partie sans casser les autres.

L'architecture générale



Frontend / Backend : pense à un restaurant. Le **frontend** est la salle et le menu (ce que le client voit). Le **backend** est la cuisine (qui prépare la réponse). Ils communiquent par une **API** : le serveur qui prend la commande et rapporte le plat.

3. Le nettoyage des données

Fichier : `scripts/clean_data.py`

Le problème

Fichier	Défaut	Conséquence
`faq_complete.json`	Erreur de syntaxe : un `],` ferme la liste des catégories trop tôt (ligne ~695)	Aucun programme ne peut lire le fichier
`emploi_du_temps.csv`	2 lignes parasites en tête, `;;;;` en fin de ligne, encodage mélangé (Windows-1252 + UTF-8)	Colonnes fausses, accents illisibles (« cr�� »)
PDF (règlement, guide)	Format non exploitable directement	Impossible d'y chercher du texte

La solution : une fonction par problème

```
def nettoyer_faq():          # répare le JSON
def nettoyer_csv():          # remet le CSV au propre
def copier_json_valides():   # vérifie les autres JSON et les copie
def extraire_pdf():          # transforme un PDF en texte
```

> **Fonction** : un bloc de code nommé et réutilisable. `def` signifie « je définis une fonction ». Bien nommées, les fonctions rendent le code lisible : on comprend le script juste en lisant leurs noms.

Réparer la FAQ

```
texte_corrige, nb = re.subn(r"\}\s*\]\s*\}\s*\],\s*\{", "}\n ]\n },\n {", texte)
faq = json.loads(texte_corrige)   # plante si le JSON est encore invalide
```

- `re.subn` cherche un **motif** (expression régulière) et le remplace. `\s*` = « des espaces ou retours à la ligne, autant qu'on veut ».
- `json.loads` sert de **vérification** : si le JSON est encore cassé, le script s'arrête immédiatement.

> **JSON** : format texte qui range les données en boîtes imbriquées. `{ }` = un **objet** (une fiche avec des champs), `[ ]` = une **liste**. **CSV** : format « tableau », une ligne par enregistrement, valeurs séparées par des virgules.

Réparer le CSV

```
def decoder_ligne(octets):
    try:
        return octets.decode("utf-8")     # on essaie l'encodage moderne
    except UnicodeDecodeError:
        return octets.decode("cp1252")    # sinon, l'encodage Windows
```

1. On cherche la vraie ligne d'en-tête (celle qui commence par `formation,`) et on ignore ce qui est au-dessus.
2. On supprime les `;` en trop avec `rstrip(";")` (« enlève ces caractères à droite »).
3. On réécrit un fichier 100 % UTF-8.

Encodage : la table de correspondance entre les lettres et les nombres stockés par l'ordinateur. Lire un fichier avec la mauvaise table transforme les accents en symboles bizarres. Le standard moderne est **UTF-8**.

try / except : « essaie ceci ; si ça plante avec cette erreur précise, fais plutôt cela ». C'est la gestion d'erreurs.

if __name__ == "__main__": « exécute ce qui suit seulement si on lance CE fichier directement ». C'est le point de départ du script.

Résultat

```
[FAQ] 1 correction(s) de structure appliquée(s)
[FAQ] 5 catégories, 152 questions
[EDT] 134 créneaux, 9 groupes
[JSON] annuaire_contacts.json valide, copié
[PDF] reglement_interieur.pdf : 46 pages -> reglement_interieur.txt
Données propres dans data/clean/
```

Pourquoi garder raw/ et clean/ séparés ? On ne modifie jamais les données d'origine. Si le nettoyage contient une erreur, on peut toujours recommencer. Le script est une « recette » que n'importe qui peut relancer : c'est la **reproductibilité**.

Pour ton rapport. Section « Préparation des données » : 3 défauts trouvés (JSON invalide, CSV mal formé, encodage mixte), comment ils ont été corrigés, et pourquoi raw/ et clean/ sont séparés.

4. La base de données

Fichier : `db/init.sql` — écrit en **SQL**, le langage des bases de données.

```
CREATE TABLE IF NOT EXISTS conversations (
    id              SERIAL PRIMARY KEY,     -- 1, 2, 3… automatiquement
    session_id      VARCHAR(64) NOT NULL,   -- identifiant ANONYME
    question        TEXT NOT NULL,
    reponse         TEXT,
    source_id       VARCHAR(32),            -- ex : "SVC001"
    categorie       VARCHAR(64),            -- ex : "services_campus"
    score           REAL,                   -- confiance de 0 à 1
    statut          VARCHAR(20) NOT NULL,   -- repondu / clarification / bloque…
    temps_reponse_ms INTEGER,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()  -- horodatage auto
);
CREATE INDEX IF NOT EXISTS idx_conversations_created_at ON conversations (created_at);
```

Mot-clé	Signification
`PRIMARY KEY`	Identifiant unique de chaque ligne
`SERIAL`	Nombre qui s'incrémente tout seul
`NOT NULL`	Colonne obligatoire
`DEFAULT NOW()`	La base remplit elle-même la date et l'heure
`CREATE INDEX`	Comme l'index d'un livre : accélère les recherches par date

Pourquoi ces colonnes ? Elles ont été choisies en pensant au **dashboard du Sprint 2**. Pour afficher le top 10 des questions, la répartition par catégorie, le taux de résolution ou le temps de réponse, il faut stocker ces informations dès maintenant. Un bon développeur anticipe la suite.

RGPD (Règlement Général sur la Protection des Données) : on ne stocke ni nom, ni email, ni adresse IP. Seulement un identifiant de session tiré au hasard (**minimisation des données**).

5. Le backend Python

C'est le cœur du projet, découpé en **5 fichiers Python** qui ont chacun un seul rôle :

```
app.py  ──utilise──▶  security.py         (la question est-elle sûre ?)
   │                  chatbot.py          (que répondre ?)
   │                      └──▶ knowledge_base.py  (chercher dans les données)
   └──utilise──▶      database.py         (enregistrer l'échange)
```

5.1 requirements.txt — les dépendances

```
flask==3.1.1            ← le framework web
flask-cors==6.0.1       ← autorise la page web à appeler l'API
scikit-learn==1.6.1     ← machine learning (TF-IDF)
psycopg[binary]==3.2.9  ← connexion à PostgreSQL
```

> **Dépendance** : du code écrit par d'autres, qu'on réutilise. Le `==3.1.1` **fige la version** : dans 6 mois, le projet s'installera exactement pareil. Sans ça, une mise à jour pourrait tout casser.

> **Framework** : une boîte à outils qui fournit la structure de base d'une application. **Flask** est un framework web Python minimaliste : quelques lignes suffisent pour un serveur qui marche.

5.2 knowledge_base.py — la base de connaissances

Rôle : charger toutes les connaissances et trouver la question connue la plus proche de celle de l'utilisateur.

Étape 1 : charger 3 sources dans un format unique

Source	Nb	Transformation
FAQ	152	Chaque question/réponse devient une entrée
Procédures	25	Question fabriquée « Comment faire : Demande de Convention de Stage ? » + étapes formatées (`formater_procedure`)
Annuaire	15	Question fabriquée « Comment contacter le service Scolarité ? » + horaires, bureau, email, téléphone

Total : **192 entrées**, toutes au même format (`id` , `question` , `reponse` , `tags` , `categorie`). Technique clé : **uniformiser des données hétérogènes** pour les traiter toutes de la même façon.

> **Comment ça a été développé.** Au début, seules la FAQ et les procédures étaient chargées. En testant « contact scolarité », le bot répondait « Comment payer mes frais de scolarité ? » **(faux)**. Il manquait l'annuaire : après son ajout, le score est passé à 0,71 **(juste)**. On découvre les manques en testant.

Étape 2 : normaliser le texte

```
def normaliser(texte):   # "Où est la Bibliothèque" → "ou est la bibliotheque"
```

Minuscules et suppression des accents : un étudiant qui tape sans accents trouve quand même la réponse.

Étape 3 : vectoriser avec TF-IDF

```
self.vectoriseur = TfidfVectorizer(analyzer="char_wb", ngram_range=(3, 5), sublinear_tf=True)
self.matrice = self.vectoriseur.fit_transform(textes)
```

> **TF-IDF** (Term Frequency – Inverse Document Frequency) : transforme une phrase en **vecteur** (une liste de nombres). Chaque nombre indique l'importance d'un morceau de mot dans la phrase. Les morceaux rares (« biblio ») pèsent lourd ; ceux qu'on trouve partout (« les ») pèsent peu.

- `fit_transform` **apprend** le vocabulaire des 192 questions et transforme chacune en vecteur. Résultat : une **matrice** de 192 lignes.
- `analyzer="char_wb", ngram_range=(3, 5)` découpe en morceaux de 3 à 5 lettres. « biblio » donne « bib », « ibl », « bli », « bibl »… C'est ce qui rend le bot tolérant aux fautes et aux abréviations.

Étape 4 : rechercher

```
def rechercher(self, question, nb_resultats=3):
    vecteur = self.vectoriseur.transform([normaliser(question)])  # question → vecteur
    scores = cosine_similarity(vecteur, self.matrice)[0]          # comparée aux 192
    meilleurs = scores.argsort()[::-1][:nb_resultats]             # top 3
```

> **Similarité cosinus** : compare deux vecteurs et donne un score de 0 (rien à voir) à 1 (identiques).

- `transform` (sans `fit`) utilise le vocabulaire **déjà appris**.
- `argsort()[::-1]` trie du meilleur au moins bon score.

> **Classe** (`class BaseConnaissances`) : regroupe des **données** (les entrées, la matrice) et des **actions** sur ces données (`rechercher` , `questions_frequentes`). `__init__` s'exécute à la création ; `self` désigne « cet objet-ci ».

5.3 chatbot.py — la décision

Rôle : décider quoi répondre, à partir des scores.

```
SEUIL_CONFIANCE  = 0.45
SEUIL_SUGGESTION = 0.30
```

Situation	Statut	Ce que fait le bot
« bonjour », « merci », « au revoir »	`small_talk`	Réponse de politesse directe
Score ≥ 0,45	`repondu`	Donne la réponse
0,30 ≤ score < 0,45	`clarification`	« Vouliez-vous dire : » + 3 boutons (= Conversation 3 du cahier des charges)
Score < 0,30	`non_compris`	S'excuse et donne le contact de la scolarité

Comment les seuils ont été trouvés. 16 questions de test, en regardant les scores. Les bonnes réponses étaient au-dessus de 0,45 ; « Comment faire ma demande ? » (ambiguë) était à 0,44 ; « Quelle est la capitale de la France ? » (hors sujet) à 0,27. Le seuil bas était d'abord à 0,25, mais la question hors sujet déclenchait des suggestions absurdes : il a été remonté à 0,30. **Les seuils se règlent avec des données, pas au hasard.**

Le champ `libelle` est une version courte de la question, affichée sur les boutons de suggestion (sinon, les boutons affichaient des phrases de deux lignes, illisibles).

Expression régulière (regex) : un motif de recherche dans du texte. Ex. : `^\s*(bonjour|salut|hello)` = « le message commence (`^`) par des espaces éventuels puis bonjour OU salut OU hello ».

5.4 security.py — la sécurité (Epic 3)

a) valider_message

Refuse ce qui n'est pas du texte, supprime les caractères invisibles, refuse un message vide ou de plus de 500 caractères.

b) detecter_injection (US-012)

```
MOTIFS_INJECTION = {
    "injection_prompt":   [r"(?i)ignore[sz]?\s+...instructions", ...],
    "injection_sql":      [r"(?i)\b(drop|delete|truncate|alter)\s+table\b", ...],
    "xss":                [r"(?i)<\s*(script|iframe|img|svg...)\b", ...],
    "injection_commande": [r"(?i)\brm\s+-rf\b", ...],
}
```

`(?i)` = sans tenir compte des majuscules. La fonction renvoie le **type** d'attaque, enregistré en base pour pouvoir compter les attaques par type dans le dashboard.

Attaque	Principe
Injection de prompt	Tenter de manipuler un assistant (« ignore tes instructions »)
Injection SQL	Glisser du code SQL pour lire ou détruire la base
XSS (Cross-Site Scripting)	Glisser du code JavaScript qui s'exécuterait dans le navigateur des autres
Injection de commande	Glisser une commande système (`rm -rf` efface des fichiers)

c) LimiteurDebit (US-013) : la fenêtre glissante

```
def autoriser(self, utilisateur):
    # 1. on oublie les requêtes de plus de 60 secondes
    # 2. s'il en reste 10 ou plus → refusé
    # 3. sinon on note l'heure de celle-ci → autorisé
```

deque (« double-ended queue ») : une liste où l'on ajoute à droite et retire à gauche très rapidement, comme une file d'attente.

Limite honnête, pour ton rapport. Le compteur est rangé par `session_id`. Un attaquant qui change d'identifiant à chaque requête contourne la limite. En production, on compterait par adresse IP ou par compte connecté. Montrer qu'on connaît les limites de son travail impressionne un jury.

5.5 database.py — parler à PostgreSQL

```
DATABASE_URL = os.environ.get("DATABASE_URL", ...)
```

Variable d'environnement : un réglage donné au programme de l'extérieur (ici par `docker-compose.yml`). On ne met jamais un mot de passe directement dans le code, puisque le code finit sur GitHub.

```
conn.execute("INSERT INTO conversations (...) VALUES (%s, %s, ...)",
             (session_id, question, ...))
```

Requête paramétrée : les `%s` sont remplis par la bibliothèque de façon sûre. C'est la protection n°1 contre l'injection SQL. Si un utilisateur tape `'; DROP TABLE conversations; --`, ce texte est stocké comme une simple question et n'est jamais exécuté.

```
except psycopg.Error as erreur:
    print(f"[DB] Impossible d'enregistrer l'échange : {erreur}")
```

Résilience : si la base tombe en panne, le chatbot continue de répondre. Les logs sont importants, mais moins que de répondre à l'étudiant.

5.6 app.py — le chef d'orchestre (l'API)

API (Application Programming Interface) : un ensemble d'adresses (des **routes**) que d'autres programmes peuvent appeler. Ici, le frontend appelle le backend.

Route	Méthode	Rôle
`/api/health`	GET (lire)	« Le serveur est-il vivant ? »
`/api/suggestions`	GET	Les questions fréquentes (US-005)
`/api/chat`	POST (envoyer)	Envoyer une question, recevoir la réponse (US-001)
`/api/logs`	GET	Les derniers échanges enregistrés (US-011)

```
base = BaseConnaissances(...)   # chargée UNE fois au démarrage
```

Si on la rechargeait à chaque question, chaque réponse prendrait une seconde. Là : **2 millisecondes**.

Le pipeline de /api/chat (l'ordre est réfléchi)

```
1. Limite de débit  → 429 si abus      (le moins coûteux d'abord)
2. Validation       → 400 si invalide
3. Anti-injection   → "bloqué" + enregistré
4. Recherche        → la réponse
5. Enregistrement   → PostgreSQL
```

Code HTTP	Signification
200	OK
400	Bad Request : ta demande est mal formée
429	Too Many Requests : trop de requêtes
500	Erreur interne du serveur

CORS (Cross-Origin Resource Sharing) : par sécurité, un navigateur interdit à une page (port 5173) d'appeler un autre serveur (port 5001), sauf si ce serveur l'autorise. C'est le rôle de `CORS(app)`.

JSON comme langage d'échange : le frontend envoie `{"message": "...", "session_id": "..."}` et reçoit `{"reponse": "...", "statut": "...", "suggestions": [...]}`.

5.7 Dockerfile du backend

```
FROM python:3.12-slim           # partir d'une image Python officielle
WORKDIR /app                    # se placer dans /app
COPY requirements.txt .         # copier D'ABORD la liste des dépendances
RUN pip install --no-cache-dir -r requirements.txt
COPY . .                        # PUIS le code
CMD ["flask", "--app", "app", "run", "--host", "0.0.0.0", "--port", "5000", "--debug"]
```

Pourquoi copier requirements.txt avant le code ? Docker garde en **cache** chaque étape. On modifie souvent le code, rarement les dépendances : avec cet ordre, Docker ne réinstalle pas tout à chaque modification. On gagne des minutes.

`--host 0.0.0.0` = « accepte les connexions venant de l'extérieur du conteneur ». Sans ça, le Mac ne pourrait pas joindre le serveur. `--debug` recharge le serveur automatiquement quand le code change.

6. Le frontend React

> **React** : bibliothèque JavaScript qui construit une page web à partir de **composants** (des morceaux d'interface réutilisables) écrits en **JSX**, un mélange de JavaScript et de HTML. **Vite** : l'outil qui fait tourner la page en développement et la recharge instantanément.

6.1 Les fichiers de configuration

Fichier	Rôle
`package.json`	L'équivalent de `requirements.txt` : dépendances (React, Vite) et commandes (`npm run dev`)
`package-lock.json`	Fige les versions exactes de toutes les sous-dépendances (généré automatiquement)
`vite.config.js`	Réglages de Vite ; `usePolling` permet au conteneur de voir les modifications faites sur le Mac
`index.html`	Page presque vide avec `<div id="root">` : c'est là que React dessine tout
`src/main.jsx`	Point d'entrée : « dessine le composant App dans #root »

6.2 App.jsx — l'interface du chat

Les états (useState) : la mémoire du composant

```
const [messages, setMessages] = useState(...)   // la liste des bulles
const [saisie, setSaisie]     = useState('')    // ce qui est tapé
const [enCours, setEnCours]   = useState(false) // le bot réfléchit ?
```

> **État (state)** : quand on appelle `setMessages(...)`, React **redessine automatiquement** l'écran. On ne manipule jamais la page à la main : on change les données, l'affichage suit.

Les effets (useEffect) : « quand X change, fais Y »

```
useEffect(() => ecrireStockage('messages', messages), [messages])
```

À chaque changement de `messages`, on les sauvegarde dans le **sessionStorage** (la mémoire de l'onglet du navigateur). C'est **US-002** : en rechargeant la page, l'historique est toujours là. Le `[messages]` final indique ce qu'on surveille.

La fonction envoyer : le cœur du composant

```
async function envoyer(texte) {
  setMessages(m => [...m, { auteur: 'user', texte: question }]) // 1. afficher la question
  setEnCours(true)                                              // 2. afficher "● ● ●"
  const reponse = await fetch(`${API_URL}/api/chat`, {...})     // 3. appeler le backend
  setMessages(m => [...m, { auteur: 'bot', texte: ... }])       // 4. afficher la réponse
  // finally → setEnCours(false)                                // 5. retirer "● ● ●"
}
```

> **async / await** : appeler un serveur prend du temps. `await` = « attends la réponse sans bloquer la page ». **fetch** = la fonction du navigateur qui envoie une requête HTTP.

> **[...m, nouveau]** : crée une **nouvelle** liste avec l'ancien contenu plus le nouveau message. En React, on ne modifie jamais une liste directement : on la remplace (principe d'**immuabilité**).

Les autres fonctionnalités

- `redemarrer()` : nouvel identifiant + historique vidé (**US-003**).
- Suggestions de départ chargées depuis `/api/suggestions` (**US-005**), affichées seulement au début.
- Bandeau RGPD (consentement), qui disparaît après « J'ai compris ».
- `crypto.randomUUID()` génère un identifiant aléatoire et anonyme (ex. `3f2a9c1e-…`).
- Accès au stockage entourés de `try/catch` : certains navigateurs le bloquent en navigation privée, l'appli marche quand même.

6.3 TexteFormate.jsx — l'affichage sécurisé

Les réponses de la FAQ contiennent `**gras**` et des lignes `- puce`. Ce composant les transforme en vrai gras et en vraies puces.

```
ligne.split(/\*\*/)   // "a **b** c" → ["a ", "b", " c"]
// les morceaux d'indice impair (1, 3, 5…) étaient entre ** → <strong>
```

> **Pourquoi pas innerHTML, plus simple ?** Parce que `innerHTML` **exécute** le HTML : si une réponse contenait `<script>`, il s'exécuterait (faille **XSS**). Ici, React affiche tout comme du texte. C'est une 2e protection en plus du filtre du backend : la **défense en profondeur**.

6.4 App.css — le style

- **Variables CSS** (`--primaire` , `--fond` …) : on change une couleur à un endroit, tout le site suit.
- **Mode sombre automatique** : `@media (prefers-color-scheme: dark)` .
- **Flexbox** (`display: flex`) : système de mise en page qui aligne les éléments ; `flex: 1` fait prendre toute la place restante à la zone de chat.
- **Responsive (US-004)** : `@media (max-width: 600px)` applique des règles spéciales sur petit écran.
- `font-size: 1rem` (16 px) dans le champ de saisie : en dessous, l'iPhone zoome automatiquement quand on tape (piège classique).

6.5 Dockerfile du frontend

Même logique que le backend : image Node, `npm ci` (installation exacte selon `package-lock.json`), puis le code, puis `npm run dev` .

7. L'assemblage : docker-compose.yml

Docker : fait tourner chaque logiciel dans une « boîte » isolée, un **conteneur**, sans l'installer sur le Mac. **Image** = la recette du conteneur. **Docker Compose** = un fichier qui décrit toutes les boîtes et les lance ensemble avec une seule commande.

```
services:
  db:       # PostgreSQL
  backend:  # Flask
  frontend: # React
```

Ligne	Explication
`image: postgres:16-alpine`	Image toute faite, pas besoin de Dockerfile
`pgdata:/var/lib/postgresql/data`	Un **volume** : les données de la base sont stockées hors du conteneur et survivent à un arrêt
`./db/init.sql:/docker-entrypoint-initdb.d/`	PostgreSQL exécute ce fichier au **premier** démarrage
`healthcheck: pg_isready`	Docker vérifie toutes les 3 s que la base est prête
`depends_on: condition: service_healthy`	Le backend attend que la base soit **vraiment** prête
`DATABASE_URL: ...@db:5432/...`	Dans Docker, les conteneurs se trouvent par leur **nom** (`db`)
`./backend:/app`	Le code du Mac est visible dans le conteneur : rechargement automatique
`./data/clean:/data:ro`	Données partagées en lecture seule (`ro`) : le backend ne peut pas les abîmer
`"5001:5000"`	Port 5001 du Mac → port 5000 du conteneur

Pourquoi 5001 et pas 5000 ? Sur Mac, le port 5000 est déjà utilisé par **AirPlay**. Beaucoup de développeurs Mac perdent une heure là-dessus.

Port : un numéro de « porte d'entrée » sur un ordinateur. Plusieurs programmes peuvent écouter sur la même machine, chacun sur son port (5173 pour la page, 5001 pour l'API, 5432 pour PostgreSQL).

8. Les petits fichiers

Fichier	Rôle
`.gitignore`	Ce que Git ne doit pas envoyer sur GitHub : `node_modules/` (des centaines de Mo, que chacun réinstalle), `__pycache__/` (fichiers Python temporaires), `.env` (secrets), `.DS_Store` (fichier caché du Mac)
`.dockerignore`	La même chose pour Docker : on évite de copier `node_modules` dans l'image
`README.md`	La première chose qu'un recruteur voit sur GitHub : commande de lancement, schéma d'architecture, structure, User Stories cochées

Git : outil qui garde l'historique de toutes les versions du code (un « Ctrl+Z » infini). **Commit** : une « photo » du projet à un instant donné, avec un message. **Push** : envoyer ses commits sur GitHub. **GitHub** : le site qui héberge l'historique en ligne (sauvegarde + vitrine).

Markdown (.md) : format texte simple pour écrire de la documentation : `# Titre`, `**gras**`, `- liste`. GitHub l'affiche joliment.

9. Les tests réalisés

9.1 Le moteur seul (avant l'interface)

16 questions variées, en regardant les scores. Extrait :

Question	Score	Réponse trouvée
horaires biblio	0,66	Horaires de la bibliothèque **OK**
c koi les horaires du resto u	0,48	Horaires du restaurant universitaire **OK**
quand commence le semestre 2	0,75	Quand commence le semestre 2 ? **OK**
contact scolarité (avant annuaire)	0,40	Comment payer mes frais de scolarité ? **FAUX**
contact scolarité (après annuaire)	0,71	Contact du service Scolarité **OK**
Comment faire ma demande ?	0,44	Clarification : Césure / Bourse / Stage **OK**
quelle est la capitale de la france	0,27	Réponse de secours **OK**

9.2 L'API dans Docker (avec curl)

Test	Résultat attendu	Obtenu
Question normale	Réponse + statut `repondu`	**OK**
Question ambiguë	Statut `clarification` + 3 suggestions	**OK**
`<script>alert(1)</script>`	Bloqué	**OK**
« Ignore toutes les instructions précédentes »	Bloqué	**OK**
Message vide	Erreur 400	**OK**
11 requêtes en moins d'une minute	Erreur 429 à la 11e	**OK**
`/api/logs`	Échanges présents en base	**OK**

> **curl** : outil en ligne de commande qui envoie des requêtes HTTP. Pratique pour tester une API sans interface.

9.3 L'interface

Un navigateur automatique (**Playwright**) a tapé des questions et pris des captures en taille ordinateur (1100 px) et mobile (390 px). Le rendu est correct dans les deux cas (US-004).

10. Les limites et la suite

Limite actuelle	Solution prévue
TF-IDF compare des **lettres**, pas du **sens** : « je veux du fric pour mes études » ne trouvera pas « bourse »	**Sentence-BERT**, qui comprend le sens (Sprint 2)
Pas de vraie détection d'intention ni d'entités	Classifieur entraîné + **NER** avec spaCy (Sprint 2)
Pas d'emploi du temps personnalisé (« mes cours lundi en B3-DATA-1 »)	Exploiter le CSV avec les entités détectées (Sprint 2)
Limite de débit contournable en changeant de session	Compter par adresse IP / compte
Serveur Flask en mode développement	Gunicorn pour le test de charge

Le programme du Sprint 2

- Classification des intentions entraînée et évaluée (objectif : précision > 85 %)
- Extraction des entités : formation, date, service, lieu
- Recherche sémantique avec Sentence-BERT
- Boutons de notation (pouce haut / pouce bas)
- Détection de la frustration et escalade vers un humain
- Dashboard Grafana

11. Lancer le projet sur ton Mac

1. **Récupérer le code** : GitHub Desktop → *File* → *Clone Repository* → `Projet-chatbot-ikrame-` → *Clone*.
2. **Ouvrir un terminal dans le dossier** : GitHub Desktop → *Repository* → *Open in Terminal*.
3. **Vérifier que Docker Desktop est ouvert** (icône de baleine dans la barre du haut).
4. **Lancer** :

   ```
   docker compose up --build
   ```

   La première fois : 2 à 3 minutes.
5. **Ouvrir le chat** : `http://localhost:5173`
6. **Voir les logs** : `http://localhost:5001/api/logs`
7. **Arrêter** : `Ctrl + C` puis `docker compose down`.

Questions à essayer

Question	Ce qu'elle montre
horaires biblio	Réponse directe malgré l'abréviation
c koi les horaires du resto u	Tolérance au langage familier
contact scolarité	Données de l'annuaire
Comment faire ma demande ?	Question ambiguë → choix proposés
`<script>alert(1)</script>`	Attaque bloquée
11 questions en moins d'une minute	Limite anti-abus

12. Glossaire

Terme	Définition
Agile / Sprint	Méthode où l'on livre des versions fonctionnelles par courtes périodes (sprints)
API	Ensemble d'adresses (routes) qu'un programme expose pour que d'autres l'appellent
async / await	Permet d'attendre une réponse (serveur) sans bloquer la page
Backend / Frontend	Le serveur (cuisine) / l'interface visible (salle du restaurant)
Classe	Regroupement de données et d'actions sur ces données
Code HTTP	Numéro standard de réponse : 200 OK, 400 mauvaise demande, 429 trop de requêtes
Commit / Push	« Photo » du projet dans Git / envoi des commits sur GitHub
Conteneur / Image	Boîte isolée qui fait tourner un logiciel / la recette de cette boîte
CORS	Autorisation donnée par un serveur à une page d'une autre origine de l'appeler
CSV / JSON	Formats de données : tableau / boîtes imbriquées
Défense en profondeur	Plusieurs protections superposées (filtre backend + affichage sûr frontend)
Dépendance	Code écrit par d'autres, réutilisé dans le projet
Docker Compose	Fichier décrivant plusieurs conteneurs, lancés ensemble
Encodage (UTF-8)	Table de correspondance lettres ↔ nombres ; UTF-8 est le standard
Expression régulière	Motif de recherche dans du texte
Fenêtre glissante	Compter les événements des N dernières secondes (limite de débit)
Framework	Boîte à outils qui fournit la structure d'une application (Flask, React)
Index (SQL)	Structure qui accélère les recherches dans une table
Injection (SQL, prompt…)	Attaque qui glisse du code ou des instructions dans un champ de texte
MVP	Minimum Viable Product : la version la plus simple qui fonctionne
Port	Numéro de « porte d'entrée » d'un programme sur une machine
Requête paramétrée	Requête SQL où les valeurs sont insérées de façon sûre (`%s`)
Résilience	Capacité à continuer de fonctionner quand une partie tombe en panne
Responsive	Interface qui s'adapte à la taille de l'écran
RGPD	Règlement européen sur la protection des données personnelles
Similarité cosinus	Score de 0 à 1 mesurant la ressemblance de deux vecteurs
SQL	Langage des bases de données relationnelles
State (React)	Variable dont le changement redessine l'écran
TF-IDF	Méthode qui transforme un texte en vecteur selon l'importance de ses morceaux
User Story / Epic	Besoin utilisateur (« En tant que… je veux… ») / groupe de User Stories

Variable d'environnement	Réglage donné au programme de l'extérieur (ex. adresse de la base)
Vecteur	Liste de nombres qui représente un texte
Volume (Docker)	Stockage qui survit à l'arrêt d'un conteneur
XSS	Attaque qui fait exécuter du JavaScript malveillant dans le navigateur d'autrui

13. Quiz d'auto-évaluation

Réponds sans regarder, puis compare avec le corrigé de la page suivante.

1. Pourquoi garde-t-on `data/raw` **et** `data/clean` ?
2. Que se passe-t-il si le score de similarité vaut 0,38 ?
3. Pourquoi écrit-on `VALUES (%s, %s)` plutôt que de coller la question dans la requête SQL ?
4. Pourquoi la base de connaissances est-elle chargée au démarrage et pas à chaque question ?
5. Dans `docker-compose.yml`, à quoi sert `depends_on: condition: service_healthy` ?
6. Pourquoi n'utilise-t-on pas `innerHTML` pour afficher les réponses ?
7. Pourquoi le backend est-il accessible sur le port 5001 et pas 5000 ?
8. Quelle est la différence entre `fit_transform` et `transform` ?
9. Pourquoi découpe-t-on les mots en morceaux de 3 à 5 lettres (`char_wb`) ?
10. Que renvoie l'API si un utilisateur envoie 11 questions en 30 secondes ? Pourquoi ce code ?

Corrigé du quiz

1. Pour ne jamais abîmer les données d'origine : si le nettoyage contient une erreur, on peut recommencer. Le script rend le nettoyage reproductible.
2. 0,38 est entre 0,30 et 0,45 : le bot n'est pas assez sûr, il répond « Vouliez-vous dire : » et propose les 3 questions les plus proches (statut `clarification`).
3. Pour se protéger de l'injection SQL : la bibliothèque insère la valeur comme du texte, jamais comme du code SQL exécutable.
4. Pour la performance : la vectorisation des 192 questions est coûteuse. Faite une seule fois, chaque réponse prend environ 2 ms au lieu d'environ une seconde.
5. Le backend attend que PostgreSQL soit réellement prêt (vérifié par `pg_isready`) avant de démarrer, sinon les premiers enregistrements échoueraient.
6. `innerHTML` exécute le HTML : du code malveillant (`<script>`) pourrait s'exécuter (XSS). React affiche le texte comme du texte : c'est la défense en profondeur.
7. Sur Mac, le port 5000 est déjà occupé par AirPlay. On relie donc le port 5001 du Mac au port 5000 du conteneur.
8. `fit_transform` apprend le vocabulaire puis transforme (fait une fois, sur la base). `transform` utilise le vocabulaire déjà appris (fait à chaque question).
9. Pour tolérer les fautes de frappe, les abréviations et les variantes (« biblio » / « bibliothèque ») : elles partagent beaucoup de morceaux de lettres.
10. Les 10 premières sont traitées, la 11e reçoit le code HTTP **429 Too Many Requests** (limite de 10 questions par minute, US-013). C'est le code standard qui signifie « trop de requêtes ».

> **Conseil.** Garde ce guide à côté de toi pendant que tu écris ton rapport et prépares ta présentation : chaque section correspond à une partie que tu peux expliquer avec tes propres mots.